Software Test Plan (STP)

Remote Lab Reservation System

Project	Remote Lab Reservation System (RLRS)
Version	1.0
Authors	Four-member Software Engineering project team
Date	31 October 2026
Status	Draft for team review

Revision History

Version	Date	Author	Change summary	Approval
1.0	31 October 2026	Four-member Software Engineering project team	Initial project-specific draft	Pending

Approvals

Role	Name	Signature / Email	Date
Project team representative	To be filled by team		Pending
Course instructor / TA	To be filled by instructor		Pending

Table of Contents

1. Introduction
2. Test Items
3. Features to be Tested
4. Features Not to be Tested
5. Test Approach / Strategy
6. Test Environment
7. Test Schedule
8. Test Deliverables
9. Roles and Responsibilities
10. Risks and Mitigation
11. Assumptions and Dependencies
12. Suspension and Resumption Criteria
13. Test Case Management and Traceability
14. Test Metrics and Reporting
15. Approvals

1. Introduction

1.1 Purpose

This plan defines the scope, strategy, environment, responsibilities, and exit criteria for testing RLRS. Tests verify that availability is shown correctly, the scheduling engine accepts only valid, non-conflicting reservations even under concurrent requests, booking rules and the reservation lifecycle are enforced, emails and reminders are sent exactly once at the right time, and access controls are enforced.

1.2 Scope

Testing covers authentication/authorization, lab and equipment management, operating hours and slots, the availability calendar, reservation creation, conflict detection, booking rules, approval, cancellation, maintenance blocks, confirmation and reminder emails, remote-access visibility, attendance, reports, audit events, the REST API, and key non-functional requirements. Physical equipment, real remote-access sessions, SSO, iCal export, SMS and production-scale email delivery are excluded unless implemented.

2. Test Items

- Authentication and role authorization
- Lab and equipment management module
- Availability service and calendar UI
- Scheduling engine (conflict detection and booking rules)
- Reservation lifecycle (approve, reject, cancel, attendance)
- Maintenance blocks
- Notification service and email job worker
- Reporting module
- Audit history
- REST API routes, services, and persistence layer

3. Features to be Tested

Requirement / feature	Test focus
RLRS-F-001 to RLRS-F-002	Valid/invalid login, logout, role restrictions, direct API access by wrong roles.
RLRS-F-003 to RLRS-F-005	Equipment create/edit/deactivate, operating hours, slot length, browse and search.
RLRS-F-006	Calendar day/week views show the correct state for every slot.
RLRS-F-007 to RLRS-F-008	Booking a free slot; overlap, containment and back-to-back boundary cases.
RLRS-F-009	Each booking rule: past time, outside hours, off-grid times, advance window, maximum duration, maximum active bookings, same-user overlap.
RLRS-F-010 to RLRS-F-012, F-018	Approval and rejection, my-reservations list, cancellation before and after cut-off, attendance marking.
RLRS-F-013	Maintenance block cancels and notifies overlapping reservations and blocks new ones.
RLRS-F-014 to RLRS-F-016	Confirmation/cancellation emails, reminder timing, single send, no reminder after cancellation, retry on SMTP failure.
RLRS-F-017 / RLRS-SR-005	Remote-access details and cancel actions limited to the reservation owner.
RLRS-F-019	Utilisation, booking and no-show report figures.
RLRS-F-021 / RLRS-SR-007	Audit event content and exclusion of secrets.

Requirement / feature	Test focus
RLRS-NF-001	Calendar and booking response time on the agreed test dataset.
RLRS-NF-002	Concurrent booking requests for the same slot.
RLRS-NF-003 / RLRS-NF-004	Persistence and reminder behaviour across restart; send-time delay.
RLRS-NF-005 / RLRS-SR-001 to SR-004, SR-006	Credential storage, authorization, validation, secrets handling and safe error responses.

4. Features Not to be Tested

- Operation of physical lab equipment or the quality of remote-access sessions; RLRS only shows access instructions.
- Delivery to real student inboxes, spam filtering and third-party email provider internals; tests stop at hand-off to SMTP (captured by a local mail catcher).
- SSO, timetable/SIS integration, iCal export, SMS or push notifications that are not part of the submitted build.
- Production infrastructure availability and load at university-wide scale.
- Third-party database, framework and calendar-library internals; test only the application's use of them.

5. Test Approach / Strategy

5.1 Test Levels

- **Unit testing:** validate the scheduling engine (overlap rule, each booking rule, slot alignment, time-zone conversion), reservation state transitions, and reminder-time calculation using a controllable clock.
- **Integration testing:** validate API/service/repository/database interactions, the database-level double-booking guard, transactional creation of email jobs, and the worker against a local SMTP catcher.
- **System testing:** run end-to-end flows (browse, book, receive confirmation, receive reminder, cancel, maintenance block) through the UI and the REST API.
- **Acceptance testing:** demonstrate the main user stories against the submitted SRS and verify the working product.

5.2 Test Types

- Functional testing of each implemented requirement.
- Boundary-value testing for time ranges: back-to-back, one-minute overlap, full containment, operating-hour edges, midnight, advance-window edge and cancellation cut-off edge.
- Concurrency testing with parallel booking requests for the same slot.
- Time-based testing of reminders using a fake/frozen clock rather than real waiting.
- API contract testing of status codes and JSON bodies for each endpoint.
- Regression testing after engine, UI, or schema changes.
- Security checks for authentication, authorization, record-level access, input validation, secrets and logs.
- Performance checks for calendar and booking using a documented dataset.
- Usability checks for calendar navigation, labels, validation feedback, and non-colour status indicators.

5.3 Entry Criteria

- A runnable build (API, UI and worker) is available and setup steps are documented.

- Test environment, test database and local mail catcher are available.
- The team has agreed the implemented requirement set, default booking rules and test data.
- Critical dependencies are installed and automated tests can be executed.

5.4 Exit Criteria

- All planned high-priority test cases have been executed.
- All critical defects are fixed or explicitly accepted by the team/instructor; no known double-booking, duplicate-reminder or authorization defect remains.
- No test run leaves two overlapping active reservations on the same equipment.
- Implemented requirements are linked to test cases in the RTM.
- Test results and deviations from SRS/SAD are documented.

6. Test Environment

Area	Plan
Hardware	Team development laptops/desktops; no lab equipment required.
Software	The same Python runtime, virtual environment, framework, database, job scheduler and calendar library specified by the project README.
Browser	Current version of a Chromium-based browser; add other browsers if required by the team.
Tools	Project unit-test framework (e.g. pytest), an API client or test client for REST calls, a fake-clock library (e.g. freezegun), a local SMTP catcher (e.g. Mailpit), GitHub Actions CI, and issue/PR tracking in GitHub.
Test data	Synthetic users for every role; two labs with different hours and slot lengths; normal and approval-required equipment; reservations in every status; maintenance blocks; a bulk set of 200 equipment items and 20,000 reservations for performance runs.
Isolation	Use a test database and mail catcher or resettable fixtures; never send test emails to real addresses or run destructive tests against shared data.

7. Test Schedule

Activity	Target window	Output
Review SRS/SAD and prepare cases	Before / during Sprint 1	Test cases and RTM updates
Unit and integration tests for implemented stories	Sprint 1 (27-31 Oct 2026)	Automated test results and defect list
End-to-end, concurrency, reminder and report tests	Sprint 2 (2-7 Nov 2026)	Execution log and resolved defects
Final acceptance run and evidence capture	By end of Sprint 2 (7 Nov 2026)	Test summary, screenshots/video evidence, final RTM
Presentation / demo readiness	From 9 Nov 2026	Final test summary and demo-ready build

Dates above follow the schedule agreed by the team: Sprint 1 from 27 to 31 October 2026, Sprint 2 from 2 to 7 November 2026, and readiness for the presentation from 9 November 2026. Adjust actual execution dates if work or submission dates change.

8. Test Deliverables

- This Software Test Plan.
- Test cases and test data/fixtures.
- Automated unit, integration and API test scripts.

- Test execution logs, CI results and captured email samples.
- Defect/issue records with reproduction steps.
- Requirements Traceability Matrix (RTM).
- Final Test Summary Report including pass/fail counts, known issues, and deviations.

9. Roles and Responsibilities

Role	Assigned person	Responsibilities
Test coordinator / QA	Team member 1 - fill in name	Maintain plan and RTM; coordinate test execution and report status.
Scheduling engine / backend developer	Team member 2 - fill in name	Add unit tests for conflict detection, booking rules and state transitions; fix engine and data-integrity defects.
Calendar UI / API developer	Team member 3 - fill in name	Verify calendar views, booking flows and API contracts; fix UI and API defects.
Notifications / CI / documentation	Team member 4 - fill in name	Test email jobs and reminders, maintain CI checks, run regression tests, collect evidence, and update final documentation.

All four members should review one another's pull requests and run relevant tests before merging. These are suggested responsibilities; replace them with the team's actual assignments.

10. Risks and Mitigation

Risk	Mitigation
Late integration causes failures	Merge small PRs frequently and run CI on every PR.
Conflict tests miss an edge case	Use a fixed table of boundary cases (before, after, touching, overlapping, containing, contained) for every rule.
Concurrency bugs are hard to reproduce	Run the parallel-booking test many times in CI against the same database engine used in the demo.
Reminder tests are slow or flaky	Use a fake clock and run the worker step directly instead of waiting.
Test emails reach real users	Point all test environments at a local mail catcher; keep real SMTP credentials out of test configuration.
Requirements changed during implementation	Record each approved deviation and update SRS, RTM, and test cases.

11. Assumptions and Dependencies

- The project build and dependencies can be installed using the repository instructions.
- Test data is synthetic and can be reset between runs.
- A local mail catcher can stand in for the real email service during testing.
- All four team members have access to the shared repository and can review/merge changes according to the team's workflow.
- Any external integration not present in the build is treated as out of scope.

12. Suspension and Resumption Criteria

Suspend a test run if the application or worker cannot start, the test database or mail catcher is unavailable, a blocker prevents more than 30% of planned cases from running, or a test risks sending email to real users or corrupting shared data. Resume when the blocking defect is fixed, the environment is restored, and a smoke test passes.

13. Test Case Management and Traceability

Test case ID	Scenario	Expected result	Requirement
TC-AUTH-01	Sign in with valid credentials	User is authenticated and sees permitted pages.	RLRS-F-001
TC-AUTH-02	Student calls an equipment-management endpoint directly	403; no data changes.	RLRS-F-002, RLRS-SR-002
TC-EQ-01	Lab in-charge creates equipment with valid fields	Equipment saved and visible in browse.	RLRS-F-003
TC-EQ-02	Book deactivated equipment	Request rejected; no reservation saved.	RLRS-F-003
TC-CAL-01	Open week view for equipment with bookings and a maintenance block	Every slot shows the correct state with a text label.	RLRS-F-006
TC-BOOK-01	Book a free 10:00-11:00 slot	201; reservation Confirmed; confirmation email captured.	RLRS-F-007, RLRS-F-014
TC-CONF-01	Book 10:30-11:30 when 10:00-11:00 is booked	409 with conflicting range; nothing saved.	RLRS-F-008
TC-CONF-02	Book 11:00-12:00 when 10:00-11:00 is booked	Accepted (back-to-back is not a conflict).	RLRS-F-008
TC-CONF-03	Book 09:00-12:00 around an existing 10:00-11:00 booking	409; containment detected.	RLRS-F-008
TC-CONF-04	Book a slot after the previous holder cancelled it	Accepted.	RLRS-F-008, RLRS-F-012
TC-RULE-01	Book a slot in the past or outside operating hours	400 naming the rule.	RLRS-F-009
TC-RULE-02	Book beyond the advance window or above maximum duration	400 naming the rule.	RLRS-F-009
TC-RULE-03	Student with the maximum active bookings makes one more	400 naming the limit.	RLRS-F-009
TC-RULE-04	Student books two different items at overlapping times	Second request rejected.	RLRS-F-009
TC-APPR-01	Book approval-required equipment, then reject without and with a reason	Pending on create; reject without reason refused; reject with reason frees slot and emails user.	RLRS-F-010
TC-CANC-01	Cancel own reservation before cut-off	Cancelled; slot free; reminder job cancelled.	RLRS-F-012
TC-CANC-02	Cancel own reservation after cut-off	409; reservation unchanged.	RLRS-F-012
TC-MAINT-01	Create maintenance block over an existing booking	Booking cancelled; holder emailed; new bookings in block rejected.	RLRS-F-013
TC-MAIL-01	SMTP unavailable when confirmation is due	Job retried; marked Failed after max attempts.	RLRS-F-016
TC-MAIL-02	Advance fake clock to reminder time and run worker twice	Exactly one reminder captured.	RLRS-F-015
TC-MAIL-03	Cancel reservation, then advance clock past reminder time	No reminder sent.	RLRS-F-015
TC-ATT-01	Mark a finished reservation as No-show	Status No-show; counted in no-show report.	RLRS-F-018
TC-REP-01	Generate utilisation report for a week	Booked hours / available hours match fixture data.	RLRS-F-019
TC-AUDIT-01	Create, approve and cancel a reservation	Audit records include actor/action/time and no secrets.	RLRS-F-021, RLRS-SR-007

Test case ID	Scenario	Expected result	Requirement
TC-CONC-01	Send 10 parallel requests for the same slot	Exactly one succeeds; others get 409.	RLRS-NF-002
TC-PERF-01	Measure 20+ week-view and booking requests on the bulk dataset	At least 90% complete within 2 seconds.	RLRS-NF-001
TC-RESTART-01	Restart API and worker with reminders queued	Data intact; each reminder sent once after restart.	RLRS-NF-003
TC-SEC-01	Inspect stored credentials, logs and repository	No plaintext password or SMTP secret found.	RLRS-NF-005, RLRS-SR-003, RLRS-SR-006
TC-SEC-02	Student cancels another student's reservation or reads its remote-access details	403; nothing disclosed or changed.	RLRS-SR-005, RLRS-F-017

14. Test Metrics and Reporting

- Execution progress = executed test cases / planned test cases x 100%.
- Pass rate = passed test cases / executed test cases x 100%.
- Track failed, blocked, and not-run cases separately.
- Track open defects by severity, age, and affected requirement.
- Report requirement coverage as requirements with at least one test / implemented requirements.
- Report double-booking incidents (target 0) and duplicate or missed reminders (target 0) across all runs.
- Publish daily or per-PR status during the sprint and a final test summary before the demo.

15. Approvals

Role	Name	Signature / Date
Test coordinator / QA	To be filled by team	
Developer representative	To be filled by team	
Project representative	To be filled by team	
Instructor / TA	To be filled if required	